

第 1 章 C++程序设计基础

本章主要介绍 C++语言及程序设计的基本概念，包括 C++简单程序格式，C++基本数据类型、运算符及表达式和简单的输入输出。通过本章的学习，可以掌握 C++语言的基础知识并能编写顺序结构的简单 C++程序，为后续的学习打下基础。

1.1 C++语言概述

1.1.1 C++语言与程序设计

语言是人类交流思想的工具。在人和计算机打交道的时候，要让计算机按人们预先安排的步骤进行工作，就要解决人和计算机进行交流的问题。人和计算机进行交流的语言，称为计算机语言。

最早的计算机语言称为机器语言。机器语言是一条条二进制代码的指令，每一条二进制指令表示一个功能，例如取数、加运算等。计算机可以直接执行机器语言而不需要转化。由计算机专业人员用指令编写的机器语言程序难读、难写、难修改。为简化机器语言，人们用符号代替二进制代码，这种便于记忆的符号语言称为汇编语言。用汇编语言写出的程序称为汇编源程序。汇编源程序上机运行时必须通过一个“汇编程序”将汇编源程序翻译成二进制指令机器才能执行。汇编语言的出现，为程序设计人员提供了很大方便。但用汇编语言编写程序同样是一件繁琐的工作，它需要程序设计人员了解计算机硬件的细节，因而影响了计算机的推广、应用。高级语言的出现为广大非计算机专业人员应用计算机提供了极大的方便。目前常用的高级语言有 BASIC、FORTRAN、PASCAL、C、C++及 JAVA 等。用高级语言编写的程序称为源程序。计算机不能直接执行源程序，必须经过“编译程序”或“解释程序”将源程序翻译成机器指令，机器才能执行。不同的高级语言有不同的编译程序或解释程序。因为计算机语言是程序设计使用的语言，所以又称为程序设计语言。

程序设计就是将解决某个问题的过程用程序设计语言描述出来，计算机按这个描述去逐步实现。不同高级语言的程序设计方法不同。因此，从程序设计方法的角度看，高级语言中的 FORTRAN、PASCAL、C 等都是面向过程的结构化程序设计语言，而 C++、JAVA 等在面向过程语言的基础上增加了面向对象的语言内容，常称为面向对象的程序设计语言。面向对象的程序设计方法被认为是最有希望、最有前途的方法。它是为适应计算机发展，特别是为操作系统等软件资源的发展而产生的。面向对象的程序设计方法是对面向过程的结构化程序设计方法的一次革命。这两者的根本区别是：在面向过程的结构化程序设计中，程序设计人员把重点放在解决某个问题的过程上；而在面向对象的程序设计中，设计人员把着眼点放在解决“什么”问题上，而不是问题的解决过程上，也就是只需关心一个对象能做什么，而不必关心对象的内部构成，从而使程序设计人员以更开阔的视野来观察问题、解决问题，使计算机的求解过程更接近人的思维活动，从而可以更充分发挥计算机系统的潜在能力。

C++是在 C 语言基础上为支持面向对象的程序设计研制的程序设计语言。C 语言的前身是 1967 年英国剑桥大学 Martin Richards 推出的 BCPL 语言。1970 年美国贝尔实验室的 Ken Thompson 以 BCPL 为基础，设计出简单而又很接近硬件的 B 语言。1973 年贝尔实验室的 D.M.Ritchie 在 B 语言的基础上保留了它简练、更接近硬件等特点，设计出了 C 语言。特别是用 C 语言成功地改写了 UNIX 操作系统，从而使 C 语言迅速得到推广，先后被移植到大、中、小型机及微机上。C 语言是一种优秀的程序设计语言。它既有高级语言的优点，也有低级语言的特点，多年来受到普遍欢迎。C++是 C 语言的扩充，它保留了 C 语言高效灵活、易于理解、可移植性好等优点，又克服了 C 语言类型检查不严格以及程序规模较大时很难查找和排除错误等缺点，同时增加了面向对象的特征，并扩充了许多新功能，使得 C++成为目前最流行的程序设计语言。

由于 C++与 C 兼容,从而使许多 C 程序代码和用 C 语言编写的大量的库函数不经修改就可以在 C++中使用。正是由于 C++是一个 C 的扩充而不是一个纯正的面向对象的语言,所以 C++既支持面向过程的结构化程序设计,又支持面向对象的程序设计。从 1980 年由美国贝尔实验室的 Bjarne Stroustrup 提出 C++以后,几经修改完善,并于 1998 年颁布了 ISO/MSIC++标准。目前,C++语言仍在不断发展之中。

1.1.2 C++语言和面向对象的程序设计

C++与 C 的最大区别是前者支持面向对象的程序设计方法。面向对象的程序设计方法简称 OOP(object oriented programming),是现代程序设计的里程碑,也是结构化程序设计以及模块化、数据抽象等方法的发展。因此,作为程序设计者,既要学习结构化程序设计方法,又要学习面向对象的程序设计方法。

在面向对象的程序设计中首先要理解“对象”一词的含义。客观世界中的任何事物都可称为对象。这些事物小到一个螺丝钉、一本书,大到一个工厂、一个学校。对象是具有某种特性和某种功能的东西。对于使用者来说,选择一个对象主要不是考虑对象内部是怎样构成的,而是考虑对象能做什么,也就是说它的功能是什么。就如同用户购买电视机时,只要知道怎样操纵各按钮就行了,至于内部构造不必关心。面向对象是一种崭新的方法,用这种方法观察世界,将客观世界看成由许多不同种类的对象组成,每种对象有特定的特征和操作,不同对象的相互作用构成了完整的客观世界。把具有共同特征(属性)和操作(方法)的对象抽象出来,形成了类。类是面向对象的程序设计中最重要概念。显然,对象是类的具体化,是某个类的一个实例。面向对象的程序主要是由对象和类建立起来的,具有封装性、继承性和多态性。

在面向对象的程序概念中,将数据和与这些数据有关的操作结合在一起,形成一个有机的整体,称为“封装”。封装是一种信息隐藏技术,通过封装将数据和其有关的行为隐蔽起来,而将一部分行为作为对外的接口。从用户或应用人员的角度看,对外的接口就是为其提供的操作功能,即为一组服务,而服务的具体实现,即对象的内部却被隐藏着。正如一台电视机给人们提供的是外部操作,而它内部的电路被隐藏起来了。在 C++程序设计中封装是由类实现的。

“继承”是面向对象的程序设计中最重要特征。继承所表达的是一种类之间的相互关系。它使某个类除了具有特有的属性和方法外,还可以继承其他类的属性和方法。在实际工作中,一个特定问题的认识和处理,往往前人已经进行过较为深入的研究和探讨,他们的研究成果后人可以利用,并且在此基础上有所发展和创造,这正是社会发展的过程。C++提供的类的继承机制允许程序设计人员在保持原有类的基础上,通过继承进行扩充成为新的类。

“多态”也是面向对象的程序设计中的一个重要特征。在 C++中,程序设计人员用向一个对象发送消息来完成一系列动作。而多态性是指不同的对象接收到相同的消息时,会产生不同的动作。C++语言支持两种多态性,即编译时的静态多态性和程序运行时的动态多态性。静态多态性通过函数重载实现,动态多态性通过虚函数机制实现。

1.2 C++程序开发过程

一个 C++系统通常由程序开发环境、语言和 C++标准库等部分组成。

在 C++程序开发环境中,一个 C++程序要经过编辑(edit)、预处理(preprocess)、编译(compile)、连接(link)、装入(load)和执行(execute)等六个阶段。以使用 Windows 下的集成程序开发环境(IDE) Microsoft Visual C++6.0 为例,除了编辑源程序要由用户自己完成外,其他阶段只需要执行一个命令就可以由 C++系统自动完成。一个在 Windows 下的典型 C++程序开发环境的各阶段及其关系如图 1.2 所示。

在编辑阶段,使用编辑程序输入、修改源程序,并存入磁盘中,文件名由用户指定。文件的

扩展名一般为.cpp(源程序)或.h(头文件)。

预处理、编译两个阶段是用一个编译(compile)命令自动先后完成的。预处理阶段处理各种预处理命令,并生成一个临时 C++源程序文件存入磁盘中,交由编译阶段进行语法检查和语义分析。如果发现语法错误,则显示尽可能多的错误信息,以使用户查找和修改错误后再次编译。如果未发现语法错误,则将生成目标程序文件并存入磁盘(扩展名一般为.obj)。

在连接阶段,连接程序把编译阶段生成的目标程序(可能有多个,一个.cpp 文件生成一个目标文件)与 C++的标准库的代码拼装成一个完整的可执行程序文件并存入磁盘(扩展名一般为.exe)。

装入程序把可执行程序从磁盘放入主存,并做好执行前的一切准备,如把代码段、各种数据段的首地址存入相关寄存器中。然后,在 CPU 的控制下执行该程序,遇到从键盘读入语句时,便等待用户输入数据并完成提交(一般按下回车键)。程序执行期间也可能存取磁盘中的外部数据文件。当程序执行中发生错误时,如除数为 0 及负数取对数或开平方时,计算机将显示出错信息。如果程序正常执行结束,则按程序中输出语句的要求输出结果。

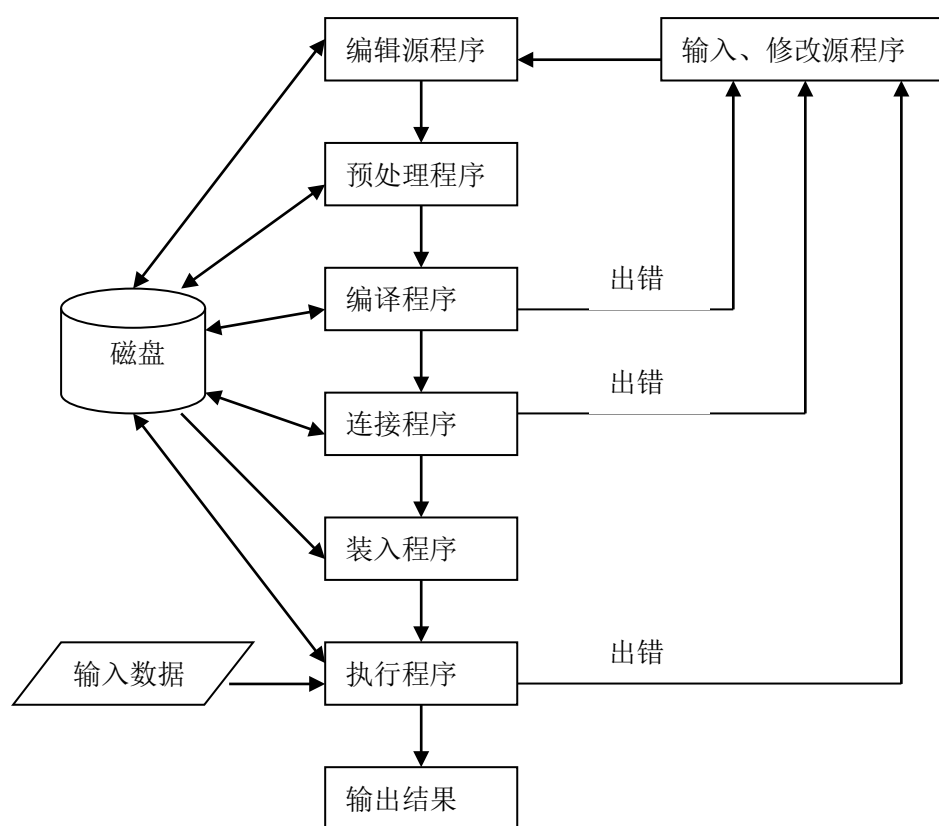


图 1.2 C++程序开发过程

在上述各阶段中都可能出现错误。其中语法错误是最常见的,这类错误可以由编译程序找出来,而且常常由于前面一个语法错误而引起后面很多语法错误。语法错误通过修改源程序很容易排除。程序运行期间也会出现致命错误而使程序中止执行(如负数取对数等)。有些错误通过仔细分析程序流程也比较容易查找和排除。最难以查找和排除的错误是属于程序逻辑上的错误(如不慎将“==”写为“=”或“<=”写为“<”和程序运行时发生的非致命错误(如数组下标越界)。因此在调试程序时,常常需要从后面的某个阶段返回到前面的编辑阶段查找错误。

1.3 C++程序实例

这里给出几个简单的 C++程序实例,以便先对 C++程序有感性认识。

例 1.1 在屏幕上输出 “Hello,you are welcome!”。

```
/* Hello program */
#include <iostream.h>

void main()
{ cout<<" Hello,you are welcome!" ;}           //你好,欢迎你!
```

执行这个程序,将在屏幕上显示如下内容:

Hello,you are welcome!

程序说明如下。

①程序中第 1 行 “/* Hello program */” 为注释。在 C++中有两种注释形式。一种是由 “/*” 开始到 “*/” 为止,中间的所有内容为注释。这种注释可以占若干行,可以出现在程序的任何地方。另一种注释形式是用 “//” 开始,“//” 后的所有内容都是注释,该注释内容直到本行结束处终止。如例 1.1 程序第 4 行的 “//你好,欢迎你!”。“//” 可出现在程序的任何行后,也可单独占一行。一般前者常用于较长内容的注释,后者多用于较短内容的注释。注释是编程者以文字的方式对程序中有关代码进行的说明或解释,为阅读程序提供了方便,但编译系统并不处理它。

②程序第 2 行#include <iostream.h>是编译预处理行。其中 “iostream.h” 为一文件名。该文件是系统库文件,文件中定义了程序中使用的 “cout” 和 “<<” 操作运算的有关信息。“cout” 和 “<<” 是系统提供的输出操作。当在程序中使用系统提供的输入、输出、数学函数运算等操作时,必须在程序开始处嵌入相关文件,称为包含文件或头文件。C++系统手册会给出各种库文件所提供的功能。预处理命令必须以 “#” 开头。因为预处理不是程序的语句,所以预处理行最后没有 “;”。这样的行必须独占一行。

③从第 3 行开始的结构

```
void main()
{
    :
}
```

是一个函数。其中 main 是个函数名。每个 C++程序必须有且只能有一个 main()函数。所有 C++程序都从 main()函数开始执行,程序中的其他操作都是由 main()函数调用或激活的。void 说明该函数的类型。void 表示该函数执行后没有任何返回值。main()函数的缺省类型为整型,即

```
int main()
```

花括号{...}之间的内容为函数体。函数所需要的语句序列或过程都在花括号之间。花括号可以不单独占一行。

④函数体内的 cout 是标准输出,它表示的标准输出设备一般是屏幕。“<<” 是个输出流插入运算符。与 cout 连用时,它表示把 “<<” 右边的内容在屏幕上显示出来。因为显示的内容是字符串,所以需要双引号将 “Hello,you are welcome!” 括起来。

⑤第 4 行字符串 “Hello,you are welcome!” 后有一分号 “;” 作为一个语句的结束标志。在 C++源程序中,一行可以写若干条语句,一个语句也可写在多行上,每个语句后必须用一个分号作为语句的结束。

⑥在 C++程序中可以从每一行的任意位置开始书写语句,不影响程序的执行。

例 1.2 编写程序从键盘任意输入 2 个数,输出这 2 个数的和。

```
#include <iostream.h>

void main()
{ int a,b,n;
  cout <<" 请输入两个数:" ; cin>>a>>b;
  n=a+b;
```

```

        cout<<" a+b=" <<n<<endl;
    }

```

程序说明如下。

①程序第 3 行的“int a,b,n;”为变量定义，定义 a、b、n 为整型变量名。在 C++程序中使用的变量都要在使用前定义。

②第 4 行“cout<<" 请输入两个数:" ;”是个输出语句，在这里起到提示作用。

③第 5 行的“cin>>a>>b;”中的 cin 代表标准输入设备，一般指键盘。“>>”是提取运算符。当它和 cin 联用时，接收用户从标准输入设备上输入的数据，并把接收到的数据赋给“>>”右边的变量。在一个 cin 语句中，“>>”可以连续使用多个。本例表示将从键盘上输入的第 1 个数送给变量 a，第 2 个数送给变量 b。从键盘上输入数据时，以空格、制表符或回车作为数据的分隔符。

④第 6 行是赋值运算，表示将从键盘上输入的 a 和 b 相加，结果存放在变量 n 中。

⑤第 7 行“a+b=”原样输出，接着将变量名 n 中的内容在输出设备上输出。第 7 行中的 endl 为换行符，用来开始一个新行。若试图将下一个内容在下一行输出，可以使用 endl。endl 也可以用“'\n’”代替，效果一样。例如：

```

    cout<<" a+b=" <<n<' \n' ;

```

执行示例如下：

请输入两个数:123 456

```

a+b=579

```

例 1.2 程序也可以写成如下形式：

```

#include <iostream.h>
void main()
{ int a,b;
  cout<<" 请输入两个数:" ; cin>>a>>b;
  int n=a+b;
  cout<<" a+b=" <<n;
  cout<<endl;
}

```

这个程序的运行结果与例 1.2 完全相同。注意这个程序中变量 n 的说明位置。C++允许根据需要随时说明新变量。

例 1.3 由两个函数组成的 C++程序。

```

#include <iostream.h>
int fmax(int a, int b)
{ if(a>b) return a;
  else return b;
}
int main()
{ int v1, v2;
  cout<<" 请输入两个数:" ; cin>>v1>>v2;
  cout<<" 两个数中较大数是:" <<fmax(v1, v2)<<endl;
  return 0;
}

```

该程序由两个函数组成，即 main()函数和 fmax()函数。在主函数 main()的 cout 中调用前面定义的 fmax()函数，将 fmax()函数执行后返回的结果在主函数的 cout 中显示输出。一个 C++程序可以由若干个函数组成，程序由 main()开始执行，一般结束也是在 main()。注意，该程序中的 main()

函数为整型，故在程序结束前返回一个整数 0。有关函数调用的详细内容将在第 3 章中介绍。

学习程序设计的过程总是由模仿开始，逐步深入，最后才能创造性地设计出自己的程序。这里的关键是要亲自动手编写程序，并且一定要上机调试、运行程序，从中发现问题、积累经验，使程序设计能力不断提高。

1.4 基本数据类型

数据是程序处理的对象。根据数据的特点，可将数据分为不同的类型。类型不同，存储方式不同，使用的场合也不同。程序中使用的数据必须属于某种数据类型。C++数据类型可以分为基本数据类型和非基本数据类型，见图 1.3。

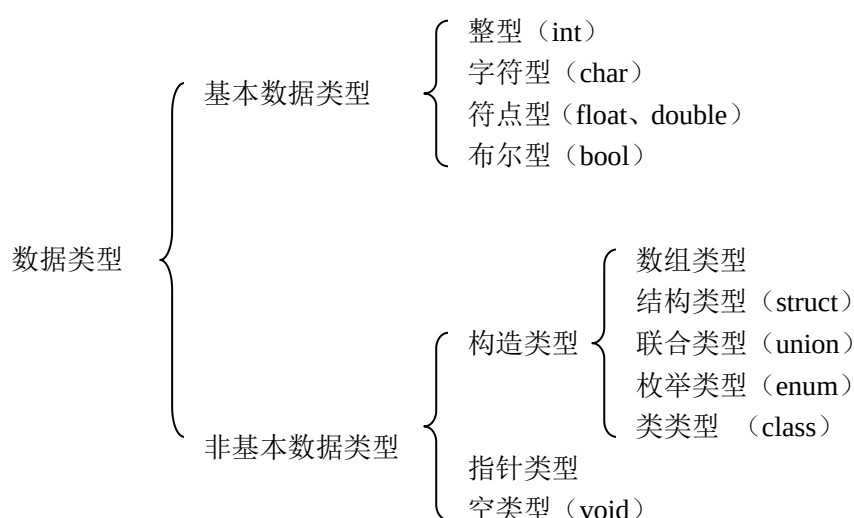


图 1.3 C++数据类型

基本数据类型是 C++系统已定义的类型。可以直接利用这些类型名来定义数据。表 1.2 给出 C++基本数据类型的名称以及在计算机内所占的位数(以字节为单位)和数据的取值范围。

表 1.2 C++基本数据类型

类型名	说明	字节	取值范围
bool	布尔型	1	true, false
[signed] chae	有符号字符型	1	-128~+127
unsigned char	无符号字符型	1	0 ~ 255
[signed] short int	有符号短整型	2	-32768~32767
unsigned short int	无符号短整型	2	0 ~ 65535
[signed] int	有符号整型	4	-2147483648~+2147483647
unsigned int	无符号整型	4	0 ~ 4294967295
[signed]long int	长整型	4	-2147483648~+2147483647
unsigned long int	无符号长整型	4	0 ~ 4294967295
float	浮点型	4	$\pm(3.4 \times 10^{-38} \sim 3.4 \times 10^{+38})$
double	双浮点型	8	$\pm(1.7 \times 10^{-308} \sim 1.7 \times 10^{+308})$
long double	长双浮点型	10	$\pm(1.2 \times 10^{-4932} \sim 1.2 \times 10^{+4932})$

对表 1.2 说明如下。

①关键字 `signed` 和 `unsigned` 以及 `short`、`long` 被称为类型修饰符。使用 `signed` 修饰的数据类型称为有符号类型，这种类型的数据可以取正数、负数和 0，`signed` 可省略。使用 `unsigned` 修饰的数据类型称为无符号类型，这种类型的数据仅能取正数和 0。

②用 `short` 修饰的数据类型的值为 `int` 类型所占空间的一半或等于 `int` 类型的值域，但目前绝大多数编译系统都是等于 `int` 类型的值域。用 `long` 修饰的 `int` 数据类型值大于或等于 `int` 类型的值域，具体值域的大小取决于具体的机器系统和所用的 C++ 编译系统。例如，在 32 位微机的 C++ 编译系统中，`int` 和 `long` 类型均占用 32 位，在 64 位机的 C++ 编译系统中它们均占用 64 位。

③用 `short`、`long`、`unsigned` 修饰 `int` 时可以省略 `int`。

④在 ISO C++ 标准中，`float` 为 7 位有效数字，`double` 为 15 位有效数字，`long double` 为 19 位有效数字。在实际使用中各系统会有所不同。

1.5 常量、变量及引用

1.5.1 常量

程序中可以直接使用的常数称为常量。常量分为整型、浮点型、字符型以及字符串常量和布尔常量。

1.整型常量

(1)C++ 可以处理不同进制的整型常量

1)十进制整数 由 0~9 数字组成的正负整数，如 0、15、-247。

2)八进制整数 以数字 0 开头的整数为八进制数。八进制数由数字 0 到 7 组成，如 015、0236。

3)十六进制整数 以 0x 或 0X 开头的整数为十六进制数。十六进制数由数字 0~9 和字母 a~f(或大写 A~F)组成，如 0x516、0x8AB、0xb2ff。

八进制和十六进制只能表示无符号整数，若写成 -015 或 -0x8AB，系统并不能将其理解为负数。

(2)C++ 可以处理长整型量和无符号整型量

当任一整型常数后跟字母 L(或 l)时，表示是长整型。若任一整型常数后跟字母 u 时，为无符号整型数。例如:12345l、7895u。

2.浮点型常量

浮点型又称实型，它由整数部分和小数部分组成。浮点型只有十进制形式。浮点型常量有两种表示形式。

1)小数形式 它由整数部分和小数部分组成，例如:3.14159，-0.55，-123.0。

2)指数形式 例如:+5.25e-8，0.5678e+05。其中 +5.25e-8 表示 $+5.25 \times 10^{-8}$ ，0.5678e+05 表示 0.5678×10^5 ，字母 e 也可以大写。浮点型数中的“+”号可以省略。

指数形式表示浮点型数时，e(E)前可以是整数或小数，但 e 后的指数部分必须是整型数。

浮点型数总是按 `double` 类型存储的，只有在数的后面加上 f 才按 `float` 类型存储，如 1.234E-6f。长双精度(long double)型常量通常在双精度数后面加上 l 或 L 表示，如 1.2345e-12L。

3.字符型常量

字符型常量是用单引号括起来的单个字符，例如 'A'、'S'、'*'、'a' 等。说明如下。

①字符型常量中的单引号只作为定界符，不是字符型常量内容。

②字符型常量具有数值，其值就是该字符的 ASCII 码值。在 C++ 中，字符型常量值可以作为整数参与运算，例如 'a' + 5 表示将 'a' 字符的 ASCII 码值与整数 5 相加，结果为字符 'f' 的 ASCII 码值(整数)。又如 '9' - 6 表示数字字符 9 的 ASCII 码值减去整数 6，结果为数字字符 '3' 的 ASCII 码值。又如 'A' + 32 结果为 'a'，'f' - 'd' 结果为整数 2。

③字符型常量可以是 ASCII 字符集中任意可打印的字符，包括空格字符。

④字符型常量中另一种字符称为转义字符。转义字符用于表示 ASCII 字符集中控制代码或某些其他功能代码。转义字符由一个“\”开始，后跟一个字符，或一个“\”后跟一个八进制或十六进制数，用单引号括起来。例如：'\n' 表示回车换行，'\\"' 表示双引号。C++转义字符见表 1.3。

表 1.3 转义字符

字符形式	功能
\ a	报警
\ b	退格
\ f	走纸（用于打印机）
\ n	换行
\ t	横向跳格（T A B 键）
\ v	垂直跳格
\ r	回车
\ '	单引号
\ "	双引号
\ \	反斜杠
\ 0	空字符
\ d d d	1 到 3 位八进制数所表示的字符
\ x h h	1 到 2 位十六进制数所表示的字符

表 1.3 中最后两行是用八进制或十六进制 ASCII 码表示的一个字符。例如，字符 a 的八进制 ASCII 码是 141，a 的十六进制 ASCII 码是 61，于是字符 a 可以表示成 '\141' 或表示为 '\x61'。换行可以用 '\012' 或 '\x0A' 表示。

4.字符串常量

用双引号括起来的一串字符称为字符串。例如：

" This is a string" , " a" , " ABC xyz\n" , " 1234"

字符串中可以包含空格、转义字符等任意字符，也可以是中文字符。双引号作为字符串的定界符，计算字符串长度时双引号不计算在内。

在 C++中字符串常量的允许长度与所用环境有关，如 VC++6.0 为 2048。编译程序在存储字符串常量时自动在字符串最后加一个 '\0' 作为一个字符串的结束标志，'\0' 占一个字节位置。因此计算字符串存储长度时总比用户写的实际字符串的字符个数多 1。例如字符串 " This is a string" 的存储长度为 17。又如 ' A' 和 " A" ，其中 ' A' 为字符常量，而 " A" 为字符串常量。前者存储长度为 1，而后者存储长度为 2。

在计算机中一个字符占一个字节，而一个汉字占两个字节。

在程序设计中，字符常量用字符变量存放，而字符串通常用字符数组存放或字符指针指向。

5.布尔常量

布尔常量仅有两个值，即 true(真)和 false(假)。

6.符号常量

除了前边介绍的在程序中直接使用的常量外，也可以为常量起一个名字，称为符号常量。符号常量在使用前必须进行说明。符号常量的说明形式为：

const 数据类型名 常量名=常量值；

或

数据类型名 const 常量名=常量值；

其中, `const` 为关键字;数据类型名为表 1.2 中的类型名。例如:

```
const int m=100;  
const float pi=3.14159;
```

表示为整数 100 起名为 `m`, 为浮点型数 3.14159 起名为 `pi`。说明后, 在程序中当用到常数 100 或 3.14159 时, 可以直接写它的符号名而不必再写该常数。

使用符号常量的好处:一是可以提高程序的可读性;二是增强程序的可维护性。如在程序中多次用到同一个常量, 要修改该常量值只要修改该符号说明中的常量值即可, 而不必逐一对程序中的常量值进行修改。使用符号常量不但提高了效率, 也可避免由于疏忽而引起的错误。

使用符号常量时的注意事项如下:

- ①符号常量在说明时一定要赋初值, 而且在程序中不能再对该符号常量重新赋值;
- ②符号常量名不要和一般变量名重名。

1.5.2 变量

在程序中可以改变值的量称为变量。

1.标识符

标识符用来为变量、符号常量、数组、函数、类型等命名。命名标识符有以下规则:必须由字母、下画线和数字组成, 而且第一个字符应是字母或下画线字符。例如, `a`、`x1`、`data5`、`count` 等为合法的标识符。标识符的长度视具体的编译系统而定。注意不要使用 C++ 的关键字(见附录 A)作为标识符。例如, `int`、`for`、`static` 等不能作为标识符。

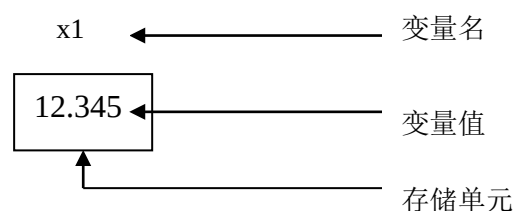


图 1.4 变量

变量名是标识符的一种。程序中使用的变量都要有一个变量名。在 C++ 中变量名区分大小写, 因此 `ex1` 和 `EX1` 是两个不同的变量名。每个变量在内存中占有一定的存储单元, 该存储单元中存放变量的值, 其关系见图 1.4。程序中的变量名代表内存中的一个存储单元, 每个存储单元都有一个地址。在程序设计中可以根据需要改变变量的值。

2.定义变量

在使用之前必须定义程序中的变量名。定义变量名时用表 1.2 中的类型名。例如:

```
char a,b,c;           //定义 a、b 为字符型变量  
int x, y;             //定义 x、y 为整型变量  
long int s1,s2;       //定义 s1、s2 为长整型变量  
float data1, data2;   //定义 data1、 data2 为浮点型变量  
double w1,w2         //定义 w1、w2 为双精度型变量  
unsigned m, n;        //定义 m、n 为无符号整型变量
```

3.变量的初始化

在定义变量的同时可以赋值, 称为变量的初始化。例如:

```
char a=' A ' ;        //定义字符型变量 a 同时给 a 赋 ' A ' 字符  
int x=0, y=12;        //定义 x、y 为整型变量, 同时给 x 赋初值 0, 给 y 赋初值 12
```

```
double w1=12.3456, w2=-0.4567e-4;//定义 w1、w2 为双精度型变量，同时赋初值
```

变量的初始化也可以写成以下形式:

```
char a(' A ' );
int x(0), y(12);
double w1(12.3456), w2(-0.4567e-4);
```

1.5.3 引用

在程序中已经定义过的变量名还可以再给它取另一个别名，这个别名就是引用(reference)。说明引用的过程就是为某个变量名建立别名的过程。说明引用的形式为:

数据类型 &引用名=变量名;

或

数据类型 &引用名(变量名);

其中变量名是已经定义过的一个变量名;“&”是说明引用的符号;数据类型是被引用变量名的类型。例如:

```
int a;
int &refa=a;
```

为变量名 a 说明了一个引用 refa。在为变量说明了一个引用后，引用名就成了这个变量的别名，在程序中使用引用名和使用变量名相同，都是对同一个存储单元的操作。例如:

```
#include <iostream.h>
void main()
{ int a(5);
  int &rea=a;
  cout<<a<<" " <<rea<<endl;
  rea=10;
  cout<<a<<" " <<rea<<endl;
}
```

输出结果为:

```
5 5
10 10
```

1.5.4 枚举类型

枚举(enumeration)类型是用户自己定义的一种数据类型。在实际应用中，有时希望用一些更具有描述性的值，而且这些值仅为有限的若干个。例如，代表星期的量有星期日、星期一、……、星期六;代表月份的量有 1 月、2 月、……、12 月。将这些可能取值的量一一列举出来，称为枚举类型。枚举类型使用时要先说明。说明枚举类型的形式为:

```
enum 枚举类型名{枚举元素};
```

其中，enum 是用于说明枚举类型的关键字，后跟枚举类型名。枚举元素用一对花括号括起来。每个枚举元素就是枚举常量值。例如:

```
enum colour{red, yellow, blue, white, black};
```

说明枚举类型，其中 colour 为枚举类型名。注意枚举类型名不是变量名，不占有内存空间。

枚举类型说明后可以定义枚举类型的变量。可以在说明枚举类型的同时定义枚举变量，也可以先说明枚举类型，另外定义枚举变量。例如:

```
enum colour{red, yellow, blue, white, black}c1, c2;
```

或

```
enum colour m1, m2;
```

分别定义 c1、c2 和 m1、m2 为枚举类型的变量。枚举变量的取值只能取说明类型时列举的元素值。例如：

```
c1=red;
c2=white;
```

枚举元素作为常量，在说明后都自动有一个整数值。该整数值按枚举元素在{}内的排列次序为 0、1、2、3……例如 colour 枚举类型中，red 的值为 0，yellow 的值为 1，……，black 的值为 4。用户可以根据需要在说明枚举类型时为枚举元素指定一个整数值，指定元素后边的各元素值按增 1 的顺序重新排列，而没有指定的元素值仍按系统自动给定的值。例如：

```
enum dd{east, west, south=10, north}x, y;
```

枚举元素 south 的值重定义为 10，则 north 的值为 11，而 east 的值仍为 0，west 值仍为 1。

枚举类型使用说明如下。

①仅能给枚举变量赋枚举元素值，若要将允许范围内的整数值赋给枚举变量，可以通过类型转换(后边介绍)，例如：

```
c1=(enum colour)3;
c2=(enum colour)1;
```

c1 中得到的是 white，c2 中得到的是 yellow。

②枚举变量进行加 1 或减 1 运算时，得到的值是当前值的前一个或后一个枚举元素。

③枚举变量进行比较关系运算时，比较的值是定义时的顺序值；

④枚举变量不能用于输入。输出时，仅输出枚举值的顺序号。

1.6 运算符与表达式

表达式由操作数和运算符组成，其中操作数可以是常量、变量及函数调用。对操作数进行的运算和处理是通过运算符进行的。

C++的表达式既可以单独作为语句使用，也可以在其他语句中作为测试的条件以及调用函数的参数使用。

在一个表达式中，运算符起着关键的作用。C++提供了丰富的运算符。这些运算符的应用范围很广，几乎所有的基本操作都可以用运算符来处理。运算符包括算术运算符、位操作运算符、赋值运算符、关系运算符、逻辑运算符、条件运算符、逗号运算符、求字节运算符、指针运算符、强制类型转换运算符等。下面分别介绍这些运算符。

1.6.1 算术运算符与算术表达式

算术运算符包括+(加)、-(减)、*(乘)、/(除)、%(取余)。使用规则如下：

①+、-、*、/与一般数学运算相同，其中“-”可作一元运算符使用，表示取负。例如 a、b 为一般变量，a+b-(a-b)*5/(a+b) 是算术表达式。

②两个整型数相除时，取商的整数部分。例如 8/5 的结果为 1，5/8 的结果为 0。实型数运算时，结果为实型数，例如，8.0/5.0 的结果为 1.6。

③%用于求两个整数相除的余数，该运算符只适用于整型数。例如，8%5 的结果为 3，而 10%5 的结果为 0，-5%3 的结果为-2，而 5%-3 的结果为 2。

④在使用二元运算符时，参加运算的两个操作数的类型可以不同。例如，一个整数和一个实数进行运算，机器会将整型数转换成实型数，再进行运算。因此在运算时，不同类型的数据要先转换成同一类型的数据，然后进行运算。转换方向见图 1.5。

图中横向的箭头表示必定的转换方向。当一个 float 型数据运算时必定先转换成 double 型，

char、short 型数据必定先转换成 int 型，再进行运算。

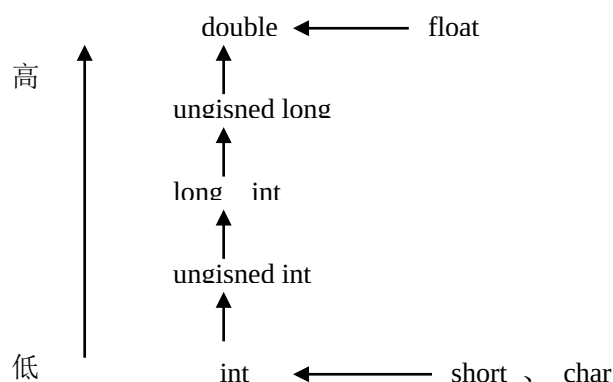


图 1.5 类型转换方向

纵向箭头表示两个不同类型的操作数运算时的转换方向。例如一个 int 型数据和一个 float 型数据运算时，都转换成 double 型后，再进行运算；一个 int 型和一个 unsigned int 型运算时将 int 型转换成 unsigned int 型后，再进行运算。注意，箭头的方向只表示数据类型的高低，运算时由低到高转换。当一个 int 型和一个 double 型数据运算时，不要理解成将 int 型先转换成 unsigned int 型，再转换成 longint 型，再转换成 double 型，而是直接将 int 型转换成 double 型。

在一个表达式中有多个运算符运算时，转换是逐个运算符进行的。例如：

```
int  a=10, b=4;
double  x=1.5;
char  c1=' A' ;
long  w=12345;
```

计算表达式(a+b)*x-w/c1 时，首先将(a+b)的运算结果 int 型转换成 double 型与 x 相乘，得到 double 型；再将 c1 转换成 long 型后与 w 相除，得到 long 型，然后将得到的 long 型转换成 double 型，与(a+b)*x 的 double 型进行相减运算，最后的结果为 double 类型。这些转换是机器自动进行的。

1.6.2 位操作运算符

位操作运算符用于对二进制数值的位进行运算。它分为逻辑位操作运算符和移位运算符，共有 6 个：

&	按位“与”，两操作数逐位求“与”
	按位“或”，两操作数逐位求“或”
^	按位“异或”，两操作数逐位相加不进位
~	按位“求反”，一元运算符，将操作数逐位取反
<<	二进制左移，将操作数左移指定位数
>>	二进制右移，将操作数右移指定位数

使用规则如下。

①参加位操作的操作数必须是整型常数或整型变量，机器自动按整型数对应的二进制值进行运算。

②逻辑位操作运算符的使用形式为：

操作数 1 & 操作数 2
操作数 1 | 操作数 2

操作数 1 $\wedge$ 操作数 2

$\sim$ 操作数

③移位运算符的使用形式为:

操作数 1 $\ll$ 操作数 2 将操作数 1 左移操作数 2 指定的位数

操作数 1 $\gg$ 操作数 2 将操作数 1 右移操作数 2 指定的位数

1.6.3 赋值运算符与赋值表达式

赋值的意思是将某个值赋予一个变量，或者说用该变量保存这个值。由赋值运算符和操作数组成的表达式是赋值表达式。C++中的赋值运算符有简单赋值运算符和复合赋值运算符。

1.简单赋值运算符

简单赋值运算符的形式为:

变量=操作数

其中的操作数可以是常量、变量和 C++任意合法的表达式。例如:

```
int a(15);
```

```
double data, s;
```

```
data=-8.1245; //赋值表达式
```

```
s=a*50+data/3; //赋值表达式
```

赋值表达式的值和类型与“=”号左边变量的值和类型相同。

2.复合赋值运算符

复合赋值运算符是将算术运算和赋值放在一起的缩写形式，包括以下几种:

$+=$ $-=$ $*=$ $/=$ $\%=$

例如:

$a+=b$; 相当于 $a=a+b$;

$a-=b$; 相当于 $a=a-b$;

$a*=b$; 相当于 $a=a*b$;

$a/=b$; 相当于 $a=a/b$;

$a\%=b$; 相当于 $a=a\%b$;

赋值运算符使用说明如下。

①赋值表达式是将赋值号右边的操作数先进行运算，然后将结果赋给(存入)左边的变量，因此赋值运算符左边必须是变量名;

②当复合赋值运算的右边是表达式时，将表达式视为一个整体，例如

```
a-=x+y;
```

相当于 $a=a-(x+y)$;

③在一个表达式中可以连续使用多个赋值运算符，运算时按由右到左的顺序进行，例如:

```
a=b=c=0;
```

相当于 $a=(b=(c=0))$

而以下使用是错误的:

```
a+=b+=c+d=12
```

1.6.4 自增和自减运算符

自增自减运算符有如下 4 种形式。

1)++i 前置自增，将 i 的值先加 1，再使用 i 的值。

2)i++ 后置自增，先使用 i 的值，然后 i 的值加 1。

3)--i 前置自减，将 i 的值先减 1，再使用 i 的值。

4)i-- 后置自减，先使用 i 的值，然后 i 的值减 1。

自增自减运算符使用说明如下。

①自增自减运算符兼有加减和赋值的功能，因此运算对象必须是变量，不能是常数或表达式，例如：

```
int a(1),b(2);
a++;
--b;
```

是正确的表达式，而“(a+b)++”和“++24”是错误的，因为常数和表达式不能被赋值。

②由自增量和自减量运算符构成的表达式单独作为一个语句使用时，采用前置增 1(或减 1)或后置增 1(或减 1)是一样的，但若和其他运算符组合使用，采用前置或后置就会产生不同的结果，例如：

```
int a, b;
a=5;
b=a++;
```

执行 b=a++时，先将 a 的值赋给 b，然后 a 加 1，执行后 a=6，b=5。若将该式改写为：

```
b=++a;
```

执行时 a 的值先加 1，然后再赋给 b，执行后，a=6，b=6。

又如：

```
int a=5, b;
b=a--+12;
```

执行后 a=4，b=17。

③自增和自减运算符++(或--)是一个整体，使用时两个+(或-)之间不要有空。此外还要注意当多于两个+或-连用时，由于++(或--)运算符是单目运算符，编译系统首先识别前两个+(或-)为自增量(或自减量)运算符，例如：对 x=a+++b，系统处理成：x=(a++)+b，因此多个+(或-)连用时，要考虑加括号，使逻辑关系清楚。若出现 x=a++++b 形式，就会产生编译错误。又如：b=(++a)+(++a)，执行结果会因为所用编译系统不同而产生不同的结果。因此在程序中应尽量避免多个+(或-)连用，必要时可以分步完成。

1.6.5 关系运算符与关系表达式

关系运算符用于比较两个数的大小，共有以下 6 个运算符：

<	小于
<=	小于或等于
>	大于
>=	大于或等于
==	等于
!=	不等于

关系运算符都是二元运算符。设 a、b 是变量，则

```
a>=(b+5)
a!=b
```

是关系表达式。

关系表达式的值是个整数值。当关系式成立时，为整数值 1；当关系式不成立时，为 0。0 和 1 可以作为整型量参加运算。例如：

```
int a, b, m, n;
a=((m+n)>=b);
```

若 $m+n$ 大于等于 b , 则 a 为 1; 否则 a 为 0。

使用关系运算符需说明以下几点。

①关系运算符两边的操作数可以是整型、浮点型、字符型、指针型及枚举型。例如, ' a ' > ' x ' 是比较两字符型常量, 实际比较的是两字符常量的 ASCII 码值。

②关系运算符可以在一个关系式中连续使用, 例如: $-1 \leq x \leq 1$ 。但如果用该式判断 x 是否在 $[-1, 1]$ 区间, 则是错误的。因为按由左到右的运算次序, 先判断 $-1 \leq x$ 是否成立, 结果为 0 或 1, 然后用 0 或 1 去判断是否 ≤ 1 , 结果永远是 1。例如 $x=3$, $-1 \leq x$ 结果为 1, 而 $1 \leq 1$, 结果为 1, 显然 3 并不在 $[-1, 1]$ 区间之内。

③注意区别 "=" 和 "=="。前者是赋值运算符, 后者是关系等于运算符。例如: 在条件语句中, $\text{if}(x=y)$ 和 $\text{if}(x==y)$ 在 C++ 中都是合法语句, 但两者的功能完全不同。前者是将 y 赋给 x , 判 x 是 0 或非 0 值; 而后者是判 x 是否等于 y 。若在程序中将 $x==y$ 误写为 $x=y$, 编译系统不会报错。如果没有及时发现, 就会产生错误的结果。

④注意 == 和 != 运算符的优先级比其他关系运算符低。

1.6.6 逻辑运算符与逻辑表达式

逻辑运算符用于对操作数进行逻辑操作。C++ 有三个逻辑运算符。它们是:

&& ? 逻辑 "与"

|| 逻辑 "或"

! 逻辑 "非" (一元运算符)

逻辑运算符的使用形式是:

操作数 1 && 操作数 2

操作数 1 || 操作数 2

!操作数

含有逻辑操作符的表达式称为逻辑表达式。逻辑表达式的值取决于逻辑关系式。当逻辑关系式成立时, 逻辑表达式的值为 1, 否则为 0。

逻辑运算符使用规则如下:

①逻辑运算符两边的操作数可以是任意基本数据类型, 只要是 0 或非 0 值即可, 任何非 0 值的数据都化为 1 参与逻辑运算。例如:

$5 \&\& 'a'$ 结果为 1

$!5$ 结果为 0

②逻辑 "与" 运算是当且仅当 "&&" 两边的操作数值都是非 0 值时, 结果为 1, 否则结果为 0。逻辑 "或" 运算是当 "||" 两边操作数中至少有一个为非 0 时, 结果为 1; 只有当两个操作数值都为 0 时, 结果才为 0。逻辑 "非" 运算是当操作数为非 0 时, 结果为 0; 当操作数为 0 时, 结果为 1。以下是正确的逻辑表达式:

$a > 0 \&\& a < 2$

$x + y > 5 || x - y \leq 1$

$!(x > y)$

要判断 x 是否在 -1 和 1 区间, 应该使用逻辑表达式:

$x \geq -1 \&\& x \leq 1$

③逻辑运算符 && 的优先级别比 || 高。在一个逻辑表达式中若有多个 && 或 || 符时, 运算总是由左到右按序进行。当能够求出整个逻辑表达式的值时将提前结束。例如:

$x \&\& (++y)$

按 && 的运算规则, 当 x 的值为 0 时, 无论 y 为何值, 该逻辑表达式的结果肯定为 0, 因此 y 的值就不再计算。同样有:

`x||y-x`

当 `x` 的值为非 0 时，逻辑表达式的结果肯定为 1，因此 `||` 后面表达式的值也不需再求。这种特性常被称为“短路操作”。例如：

```
int x=2,y=1;
cout<<(x>y||(++y)&&(++x))<<endl;//由于 x>y 为 1，(++y)&&(++x)被短路
cout<< x<<y<<endl;
```

输出结果为：

```
1
2 1
```

1.6.7 其他运算符

C++还有以下几种运算符。

1.条件运算符与条件表达式

条件运算符是三元运算符，它需要 3 个操作数，形式为：

`e1?e2:e3`

其中 `e1` 一般为关系或逻辑式；`e2` 和 `e3` 为任意表达式。

条件运算符的功能是：当 `e1` 为非 0 值时，取 `e2` 的值为该条件式的值；当 `e1` 为 0 值时，取 `e3` 的值为该条件式的值。由条件运算符组成的表达式为条件表达式。例如：

```
a=x>y?x+y:x-y;
```

将一个条件表达式的值赋给 `a`。当 `x` 大于 `y` 时，将 `x+y` 的值赋给 `a`，否则将 `x-y` 的值赋给 `a`。

2.逗号运算符与逗号表达式

用逗号“`,`”将几个表达式隔开的式子称为逗号表达式。该表达式从左到右计算各表达式值，并以最后一个表达式的值作为逗号表达式的值。逗号表达式的形式为：

`e1, e2, ..., en`

例如，赋值运算

```
x=(a=1, b=3, ++b, a+b);
```

的结果为 5。

3.求字节运算符

求字节运算符是个一元运算符，形式为：

`sizeof(e)`

其中，`e` 可以是任意类型的变量、类型名或表达式。该运算符的作用是求 `e` 在内存中占用的字节数。例如：

```
int a, b, i;
double x;
a=sizeof(x);
b=sizeof(float);
i=sizeof(a*x);
```

其中，`a` 中得到的是 `x` 变量占用的字节数；`b` 中得到的是单浮点 `float` 类型占用的字节数；`i` 中得到的是表达式 `a*x` 的值占用的字节数。

4.类型转换符

当一个表达式中有几种不同类型的数据参加运算时，编译系统自动按由低到高的次序进行类型转换。也可以利用类型转换符强制转换成所需要的类型。类型转换符的使用格式是：

(类型名)表达式

或

类型名(表达式)

两种形式是等效的。作用是将“表达式”的值转换成“类型名”所指定的类型。例如:

`(int)(x+y)`

表示将 `x+y` 的运算结果转换为整型。该式也可写成

`int(x+y)`

效果是一样的。若写成

`(int)x+y`

表示只转换 `x` 成整型, `y` 不转换。

转换时若整型转换为字符型, 则去掉多余的高位;长整型转换为一般整型时, 去掉多余高位;浮点型转换成整型时, 去掉小数部分;双精度型转换成单精度型时, 按单精度型位数舍入。因此这类转换是一种不安全的转换, 数据的精度会受到影响。

1. 6. 8 运算符的优先级和结合性

1)优先级 优先级是指不同操作符之间进行操作的先后顺序。当几种不同的运算符出现在一个表达式中时, 运算符的优先级就很重要。在同一个表达式中, 优先级别高的先计算, 优先级别低的后计算。

2)结合性 当连续几个相同级别的运算符进行计算时, 就要考虑结合性。结合性不同, 运算的次序就不一样, 有的是由左到右逐个运算, 有的是由右到左逐个运算。

表 1.4 给出不同类型运算符及它们的优先级和结合性。其中优先级由高到低的次序用 1、2、3……表示。全体一元运算符、三元运算符和赋值运算符都是从右向左结合的, 其余都是从左向右结合的。

表 1.4 C++运算符及其优先级与结合性

优先级	运算符	功能说明	结合性
1	::	作用域符	从左至右
2	。、-> () [] .、*、.->	成员选择 括号、函数调用 数组下标 成员指针选择符	从左至右
3	sizeof ++、-- ~ ! +、- & * new、delete (类型)	求所占内存字节数 增、减 1 运算符 按位取反 逻辑非 正号、负号 取地址 取内容 动态分配、释放空间 类型转换	从右至左
4	*, /, %	乘法、除法、求余	从左至右
5	+, -	加法、减法	从左至右
6	<<, >>	按位左移、右移	从左至右
7	<, <=, >, >=	小于、小于等于、大于、大于等于	从左至右
8	==, !=	等于、不等于	从左至右
9	&	按位与	从左至右
10	^	按位异或	从左至右
11		按位或	从左至右
12	&&	逻辑与	从左至右
13		逻辑或	从左至右
14	?:	条件运算符（三目运算符）	从右至左
15	=, +=, -=, *=, /=, %=, <<=, >= &=, =, ^=	赋值运算符	从右至左
16	,	逗号运算符	从左至右

1.7 基本输入、输出

输入、输出又称 I/O(Input/Output)操作,是程序设计中必不可少的操作。输入是指在程序运行时由输入设备(常指键盘)向程序提供数据;输出是指将程序的运行结果在输出设备(常指显示器)上显示。

C++没有提供专门的输入、输出语句,输入、输出功能通过输入、输出流和函数实现。将数据从一个对象到另一个对象的流动抽象为流(Stream)。输入、输出时把一个个数据都看成是字节流。当用键盘键入字符时,就可以认为字符从键盘流入程序的变量中;当输出字符时,也可以认为数据从程序流到输出设备。

在前面程序中已经使用的 cin 和 cout 是预定义的输入、输出流对象。cin 是输入流对象的名字,

其后的“>>”是提取操作符，作用是从标准输入设备(即键盘)的输入流中提取若干个字符送到程序中指定的变量。cout 是输出流对象的名字，其后的“<<”是流插入操作符，作用是将程序中需要输出的内容送到标准输出设备(即显示器)输出。cin 和 cout 在〈iostream.h〉文件中定义。因此使用时在程序开始处要写入相应的预编译命令，即

```
#include <iostream.h>
```

1.cin 的使用形式

cin 的使用形式如下:

```
cin>>变量 1>> 变量 2>>...>>变量 n;
```

使用 cin 一次可以输入多个值，即可以输入任何基本类型的数据。例如:

```
int a,b,c;
double x1,y1;
cin>>a>>b>>c;
cin>>x1>>y1;
```

可以将以上两个 cin 用一个实现，即

```
cin>>a>>b>>c>>x1>>y1;
```

执行时逐个从键盘输入数据，用空格、制表符或回车作为输入的两个数据之间的分隔。当输入数据的个数多于 cin 后变量名的个数时，多送的数据没用;当输入数据的个数少于 cin 后变量名的个数时，系统等待继续输入。

2.cout 的使用形式

cout 的使用形式如下:

```
cout<<表达式 1<<表达式 2<<...<<表达式 n;
```

其中的每个表达式可以是常量、变量、表达式、字符串、函数调用等。当是表达式时，输出表达式结果的值。若是函数调用，则输出函数调用后的结果。当是字符串时，将字符串原样输出。例如:

```
int n=10;
cout<<3.14159<<" " <<n<<' \a' <<' \n' ;
```

输出结果为:

```
3.14159 10
```

在输出的 3.14159 和 10(n 的值)之间有一个空格，输出 10 后机器响铃，最后回车换行。注意:当用 cout 输出一个表达式的值时，表达式最好用括号括起来，因为当表达式中运算符级别低于<<时，编译会出错。

需要说明的是，1998 年颁布的 ISO C++ 语言标准对 C++ 语言的标准库进行了部分修改，其中包括重新命名定义输入输出功能的头文件，命名的方法是将原来头文件名中的扩展名“·h”去掉，例如将 iostream.h 改为 iostream 从而成为一个新的头文件名。此外，新标准中还引入了名字空间(namespace)的概念，目的是为了一个程序不同模块中相同名称所引起的命名冲突。因此修订的 C++ 标准库也遵守该原则，即为了减少用户程序中的标识符与标准库中的标识符产生冲突，其中包括 iostream 在内的大部分头文件被放到了 std 名字空间。因此用户可以用以下方法引用名字空间:

1) 使用关键字 using 将 std 名字空间中的标识符全部引入到用户程序中。具体用法是在头文件 include 预处理行的下一行写上语句:

```
using namespace std;
```

这样就可以在程序中继续使用 cin 和 cout 进行输入输出操作。

例 1.4 输入、输出示例。

```
#include <iostream.h>
```

```

void main()
{ int age;
  cout<<" Enter your age:" ; cin>>age;
  cout<<" your age is " <<age<<endl;
}

```

使用新标准将程序改为:

```

#include <iostream>           //新标准的头文件
using namespace std;         //引入 std 名字空间
void main()
{ int age;
  cout<<" Enter your age:" ; cin>>age;
  cout<<" your age is " <<age<<endl;
}

```

上述两程序实现的功能完全相同。输出结果为:

```

Enter your age: 20
your age is  20

```

其中 `cout<<" Enter your age:"` 语句起到提示作用。

2) 使用范围限定符 “::” 在用户程序中显示地说明属于 `std` 名字空间地标识符。具体用法是在 `cin` 和 `cout` 前加 `std::`。例如将上述程序用限定符以适应新标准:

```

#include <iostream>           //新标准的头文件
void main()
{ int age;
  std::cout<<" Enter your age:" ; std::cin>>age;
  std::cout<<" your age is " <<age<<std::endl;
}

```

另外换行符 `endl` 也在 `std` 名字空间中说明。

3.在输入、输出流中使用控制符

上面介绍的 `cin` 和 `cout` 输出数据时使用的是默认格式。当用 `cout` 输出一个整型数时, 原样输出该整数的值; 输出一个 `float` 型数或 `double` 型数时, 默认提供 6 位有效数字。

例 1.5 默认格式输出。

```

#include <iostream>
using namespace std;
int main()
{ int a=123;
  long int b=1234567;
  float x1=12.34, x2=23.456789;
  double y=12.34567890123;
  cout<<" a=" <<a<<' \n' <<" b=" <<b<<endl;
  cout<<" x1=" <<x1<<" " <<" x2=" <<x2<<endl;
  cout<<" y=" <<y<<endl;
  return 0;
}

```

输出结果为:

```

a=123

```

```

b=1234567
x1=12.34  ?x2=23.4568
y=12.3457

```

可以使用 C++提供的控制符控制输出数据的格式。常用的几个控制符如下:

```

dec  转换为十进制数输入/输出
hex  转换为十六进制数输入/输出
oct  转换为八进制数输入/输出
setw (int)      设置输出的宽度
setprecision (int)  设置浮点数输出的有效数字位数
setfill (char)   设置填充字符
endl            ?插入换行符

```

注意:使用以上格式控制符时, **double** 类型最多可提供 15 位有效数字, 并且在程序的开头除要写上#include <iostream.h>头文件外, 还要加上#include <iomanip.h>头文件。

例 1.6 使用格式控制符输出。

```

#include <iomanip>
#include <iostream>
using namespace std;
void main()
{ int x=24;
  double y=12.3456789;
  cout<<dec<<x<<" ";
  cout<<hex<<x<<" ";
  cout<<oct<<x<<" \n ";
  cout<<dec; //以下仍按十进制输出
  cout<<setw(8)<<x<<" ," <<x<<endl;
  cout<<setw(8)<<setfill(' ')<<x<<endl;
  cout<<setprecision(5)<<y<<endl;
}

```

输出结果为:

```

24 18 30
    24,24
*****24
12.346

```

程序说明如下:

①程序中 dec、hex 和 oct 分别表示将 x 值按十进制、十六进制和八进制形式输出, 程序执行时自动将 24 转换输出。输出的十进制数为 24, 十六进制数为 18, 八进制数为 30。输出十进制数时 dec 可省略。hex 和 oct 也可用于输入, 使用形式与输出相同。

②cout<<setw(8)<<x<<" ," <<x<<endl 语句中的 setw(8) 用于设置输出的宽度。这里表示按十进制形式留出 8 位字符位置, 不足 8 位时前边填加空格。由于 setw()只对其后的第一个数据起作用, 因此在输出第二个 x 时, setw(8)不起作用, 按默认格式输出 24, 前面没有空格。

③cout<<setw(8)<<setfill(' ')<<x<<endl 语句表示按 8 位字符位置输出 x, 空格的位置用“*”填充。

④setprecision(5) 表示输出 y 时的有效数字为 5 位, 在第 4 位小数上四舍五入。因此舍入后输出的 y 值为 12.346。

有关输入输出的更多内容见第 10 章 I/O 流。

例 1.7 从键盘输入一个半径值，求该半径圆的周长和面积。

```
#include <iostream>
using namespace std;
const double pi(3.14159);
void main()
{ double r,a,c;
  cout<<" 输入半径值:" ; cin>>r;
  c=2*pi*r;
  a=pi*r*r;
  cout<<" 周长=" <<c<<endl;
  cout<<" 面积=" <<a<<endl;
}
```

执行示例如下:

```
输入半径值: 2.5
周长=15.708
面积=19.6349
```

1.8 例题

例 1.8 算术运算符和增量、减量运算符使用。

```
#include <iostream>
using namespace std;
void main()
{ int a,b;
  float x,y;
  a=-5; b=10;
  x=1.75;?y=2.34e-03;
  cout<<" a=" <<a<<" ?" <<" b=" <<b<<endl;
  cout<<" a+b=" <<a+b<<endl;
  cout<<" a=" << ++a<<" ?" <<" b=" <<--b<<endl;
  cout<<" a=" << a++<<" ?" <<" b=" << b--<<endl;
  cout<<" a*b=" <<a*b<<endl;
  cout<<" a%b=" <<a%b<<endl;
  cout<<" x+y=" <<x+y<<endl;
}
```

输出结果为:

```
a=-5 b=10
a+b=5
a=-4 b=9
a=-4 b=9
a*b=-24
a%b=-3
x+y=1.75234
```

例 1.9 关系运算符使用。

```
#include <iostream>
using namespace std;
void main()
{ int a,b;
  cout<< "输入两个正整数:" ;
  cin>>a>>b;
  cout<<" a>b:" <<(a>b)<<' \n' ;
  cout<<" a<=b:" <<(a<=b)<<' \n' ;
  cout<<" a==b:" <<(a==b)<<' \n' ;
  cout<<" a!=b:" <<(a!=b)<<' \n' ;
}
```

程序执行时屏幕显示:

输入两个正整数:5 6

输出结果为:

```
a>b:0
a<=b:1
a==b:0
a!=b:1
```

例 1.10 使用条件运算符，求三个数的最大值。

```
#include <iostream>
using namespace std;
void main()
{ int a,b,c,m;
  cout<<" 输入 3 个数:" ;
  cin>>a>>b>>c;
  m=a>b?a:b;
  cout<<" 最大值是:" <<((m>c)?m:c)<<endl;
}
```

运行程序时:

输入 3 个数:12 79 36

输出结果为:

最大值是:79

例 1.11 逻辑运算符使用。注意逻辑运算的“短路”。

```
#include <iostream>
using namespace std;
void main()
{ int a,b,c,x,y;
  a=1; b=2; c=0;
  cout<<a++<<endl;
  cout<<(a&&b||!c)<<endl;
  cout<<b/++a<<endl;
  x=++a||++b&&++c;           //逻辑运算“短路”，b 和 c 不计算
  cout<<a<<b<<c<<x<<endl;
```

```

    a=b=c=-1;
    y=++a&&++b&&++c;//逻辑运算“短路”，b和c计算
    cout<<a<<b<<c<<y<<endl;
}

```

输出结果为:

```

0
1
0
4 2 0 1
0 -1 -10

```

例 1.12 混合使用运算符。

```

#include <iostream>
using namespace std;
void main()
{ int a=4,b=3,c=2,d=1;
  int x, y=0;
  x=a<b?a+1:c<d?c+1:d+1;
  cout<<x<<endl;
  cout<<(a+b,b+c,c+d)<<endl;
  y+=a+=b+=c+d;
  cout<<y<<endl;
}

```

输出结果为:

```

2
3
10

```


练习 1

1.回答以下问题:

- (1)C++程序中两种注释形式分别是什么?
- (2)C++有哪些基本数据类型?
- (3)字符常量和字符串有什么区别?
- (4)转义字符的作用是什么?
- (5)开发一个 C++程序需要哪几个阶段,各阶段的作用是什么?
- (6)关系运算符和赋值运算符哪个优先级高?
- (7)为什么自增 1 和自减 1 运算符只能用于变量?
- (8)举例说明什么是逻辑运算中的“短路”?
- (9)一个表达式的类型如何确定?
- (10)引用的作用是什么?

2.判断以下常量的正误,指出正确常量的类型。

0 3.25741 ' 5' " abc..." -0.345e150 .12345e-10
0234 0x87AB ' yy' a25+36 0Xabff 0569

3.判断下列变量名哪些是正确的。

ax int SS1 36A a+b Text c/dab _tt30
static array total_8 do www number x007 class

4.设 a、b 为整型变量, a=5, b=3, 写出下列各表达式的值。

- (1)a++
- (2)a=b
- (3)a&&b
- (4)b<=2&& a>=0
- (5)a>=5|| b<=5
- (6)b||a&&0
- (7)x=(a++)+(++b)
- (8)a+=(a+b)
- (9)a%3-1
- (10)x=++a/b--
- (11)a<b?a+2:b+2
- (12)!(a-b)

5.选择填空题。

- (1)设有定义:int x=0,y=5;表达式:y+=x/5+4;的值是 ____ 。
A)0 B)5 C)4 D)9
- (2)设有定义:int m=8, i, j;double x=1.42, y=5.2;以下符合 C++语法的表达式是 ____ 。
A)x+y %=m ?B)(m-2)++ C)i=j*5=3 ?D)m+=m-=2*(j=3)
- (3)设有定义:int a=5, b=2; 以下值为 1 的表达式是 ____ 。
A)!(b==a/2) B)b!=a ?C)a!=b||a>=b D)a>0&&b<2
- (4)设有定义:int x=2, y=3, z=4;以下能正确表示 1xyz 的表达式是 ____ 。
A)1/x*y*z B)1/(x*y*z) C)1.0/x/y/z ?D)1/x/y/(float)z
- (5)设有定义:int a,b;表达式 (a=3,b=5,a>b)?a++:b++, a+b 的值是 ____ 。
A)3 B)8 ?C)9 ?D)10
- (6)设有定义:char c1=' a' ,c2=' A' ;表达式 c1<c2? c1:c2+32 的值是 ____ 。

A)0 B)1 C)' a' ?D)' A'

6.写出下列程序的运行结果。

(1) #include <iostream >

```
using namespace std;
void main()
{ int m(1) , n(2), k;
  k=++m;
  cout<<" k=" <<k<<endl;
  k=m+n++;
  cout<<m<<n<<k<<endl;
  k=-n-m;
  cout<<m<<n<<k<<endl;
  k=(m>=n);
  cout<<k<<endl;
}
```

(2) #include <iostream>

```
#include <iomanip>
using namespace std;
void main()
{ int a(2), b(3);
  double c,d;
  c=-0.5;
  d=8.123456;
  cout<<setw(3)<<a+b<< endl;
  cout<<setw(10)<<setfill(' *' )<<a*b<<endl;
  cout<<setprecision(5)<<d<< endl;
  cout<<setprecision(4)<<c*d<< endl;
}
```

(3) #include <iostream>

```
using namespace std;
int main()
{ float x=12.345;
  int y=100;
  cout<<x*y<<endl;
  y=x*y;
  cout<<y<<endl;
  return 0;
}
```

(4)#include <iostream>

```
using namespace std;
void main()
{ int a,b,c,sum;
  cin>>a>>b>>c;           //执行时输入 5、6、7
  int &resum=sum;
```

```
    resum=a+b+c;  
    cout<<sum<<endl;  
}
```

7.编写下列程序:

- (1)任意输入一个整数值，输出该数的 10 倍。
- (2)任意输入 2 个数，输出其中的较小者。
- (3)输入半径 *radius*，输出该半径对应的圆的周长和面积。
- (4)将 10000 秒化成小时、分、秒。
- (5)将输入的一个三位正整数的各位数字分 3 行输出。例如，输入 456，输出

6

5

4